ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO THỰC HÀNH

LAB 02: LẬP TRÌNH ASSEMBLY CƠ BẢN

Basic Assembly Programming – Lập trình hợp ngữ x86

Nhóm: Nhóm 4
Sinh viên 1: Bùi Thái Sơn – 25521588
Sinh viên 2: Võ Quốc Thịnh – 25521787
Sinh viên 3: Lê Thành Đạt – 25520270

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Chi tiết các bài tập và giải thích mã nguồn **1**

1.1 Bài 2.1: In ra độ dài của chuỗi cho trước (tối đa 9 ký tự) 1

1.2 Bài 2.2: Tính số dư của tổng 4 số có 2 chữ số khi chia 8 1

1.3 Bài 2.3: Mã hóa Caesar trên một chuỗi ký tự (Cơ bản + Nâng cao 1 & 2) 3

1.4 Bài 2.4: Đếm số lượng bội của 10 trong khoảng [x, y] 4

1.5 Bài 2.5: Kiểm tra tuổi đoàn dựa vào năm sinh (Cơ bản + Nâng cao) 6

1.6 Bài 2.6: Kiểm tra năm nhuận (Cơ bản + Nâng cao) 8

2 Minh chứng kết quả thực thi **12**

1 Chi tiết các bài tập và giải thích mã nguồn

1.1 Bài 2.1: In ra độ dài của chuỗi cho trước (tối đa 9 ký tự)

Yêu cầu đề bài: Cho trước một chuỗi ký tự `msg` được định nghĩa trong `.section .data` (độ dài tối đa 9 ký tự). Hãy viết chương trình hợp ngữ tính độ dài chuỗi (không tính ký tự null kết thúc chuỗi `\0`) và in giá trị này ra màn hình kèm ký tự xuống dòng.

Giải thích mã nguồn: Chương trình xác định trước độ dài chuỗi cố định `msg` ngay ở giai đoạn hợp dịch thông qua biểu thức `msg_len = . - msg - 1`, lấy vị trí hiện tại trừ địa chỉ đầu chuỗi và trừ bỏ 1 byte null `\0`. Giá trị số này được cộng với `0x30` để chuyển đổi trực tiếp sang ký tự chữ số ASCII rồi lưu sẵn cùng mã xuống dòng `0x0a` trong vùng nhớ `output`. Khi khởi chạy, chương trình chỉ cần nạp địa chỉ buffer vào `%ecx`, độ dài 2 bytes vào `%edx` và gọi `sys_write` (`%eax = 4`) ra `stdout` (`%ebx = 1`) qua ngắt `int $0x80`, sau đó thoát tiến trình an toàn bằng `sys_exit`.

Mã nguồn chương trình (`bai2.1.s`):

```
.section .data
msg:
    .string "IloveCR7"
msg_len = . - msg - 1 # Lay dia chi hien tai - dia chi msg - 1 (tru byte null)

output:
    .byte msg_len + 0x30 # Cong 48 (0x30) de chuyen thanh ky tu ASCII
    .byte 0x0a           # Ky tu xuong dong '\n'

.section .text
    .globl _start
_start:
    movl $output, %ecx   # Dia chi buffer can xuat
    movl $0x2, %edx      # Do dai xuat ra: 2 bytes
    movl $0x1, %ebx      # File descriptor: stdout (1)
    movl $0x4, %eax      # System call number: sys_write (4)
    int $0x80            # Goi kernel xuat chuoi

    movl $0x1, %eax      # System call number: sys_exit (1)
    int $0x80            # Thoat chuong trinh
```

1.2 Bài 2.2: Tính số dư của tổng 4 số có 2 chữ số khi chia 8

Yêu cầu đề bài: Nhập lần lượt 4 số nguyên có 2 chữ số ($10 \leq x \leq 99$) từ bàn phím. Tính tổng 4 số đã nhập và in ra màn hình số dư của tổng khi chia cho 8.

Giải thích mã nguồn: Chương trình tách tác vụ nhập liệu thành thủ tục con `nhapso` và gọi 4 lần liên tiếp để nhận lần lượt 4 số từ người dùng qua `sys_read`. Mỗi số đọc được gồm 2 ký tự số được chuyển đổi về giá trị nguyên bằng cách trừ 48, lấy chữ số đầu nhân 10 rồi cộng với chữ số sau trước khi cộng dồn vào biến `ans`. Sau khi hoàn tất nhập liệu, thủ tục `inkq` xóa thanh ghi `%edx`, nạp tổng vào `%eax` và thực hiện phép chia không dấu 64-bit cho 8 bằng lệnh `divl %ecx`. Phần dư nằm trong `%edx` được cộng thêm 48 thành mã ASCII và in ra màn hình cùng chuỗi thông báo qua `sys_write`.

Mã nguồn chương trình (`bai2_2.s`):

```
.section .data
nhapvao:
    .string  "Enter a number (2-digit): "
kq:
    .string "Remainder: "
newline:
    .byte 10
ans:
    .int 0

.section .bss
```

```
    .lcomm number1, 4

.section .text
    .globl _start

jmp _start

nhapso:
    movl $4, %eax        # sys_write in loi nhac
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $26, %edx
    int $0x80

    movl $3, %eax        # sys_read doc 3 bytes tu STDIN
    movl $0, %ebx
    movl $number1, %ecx
    movl $3, %edx
    int $0x80

    movzbl number1, %eax    # Doc chu so hang chuc
    subl $48, %eax          # Chuyen thanh gia tri nguyen
    imull $10, %eax         # Nhan voi 10

    movzbl number1+1, %ebx  # Doc chu so hang don vi
    subl $48, %ebx
    addl %ebx, %eax         # Tong gia tri cua so vua nhap

    addl %eax, ans          # Cong don vao bien tong ans
    ret

inkq:
    movl ans, %eax       # Nap tong vao %eax
    xorl %edx, %edx      # Xoa %edx de chuan bi chia 64-bit
    movl $8, %ecx        # So chia = 8
    divl %ecx            # Chia: thuong o %eax, so du o %edx

    movl %edx, ans
    addl $48, ans        # Chuyen so du sang ky tu ASCII

    movl $4, %eax        # In chuoi thong bao "Remainder: "
    movl $1, %ebx
    movl $kq, %ecx
    movl $11, %edx
    int $0x80

    movl $4, %eax        # In ky tu so du
    movl $1, %ebx
    movl $ans, %ecx
    movl $1, %edx
    int $0x80

    movl $4, %eax        # In dau xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret

_start:
    call nhapso
    call nhapso
    call nhapso
    call nhapso
    call inkq

    mov $1, %eax         # sys_exit
    int $0x80
```

1.3 Bài 2.3: Mã hóa Caesar trên một chuỗi ký tự (Cơ bản + Nâng cao 1 & 2)

Yêu cầu đề bài:

- **Cơ bản:** Nhập một chuỗi ký tự và số bước dịch n ($0 \leq n \leq 9$), xuất ra chuỗi đã được mã hóa Caesar tương ứng.
- **Nâng cao 1:** Xử lý xoay vòng chuẩn khi ký tự sau khi dịch vượt quá `'z'` hoặc `'Z'` (ví dụ: với $n = 3$, `'z'` chuyển thành `'c'`).
- **Nâng cao 2:** Hỗ trợ đồng thời cả chữ in hoa (`'A'`–`'Z'`) và chữ in thường (`'a'`–`'z'`). Tất cả ký tự số, ký tự đặc biệt, khoảng trắng được giữ nguyên không đổi. Chuỗi đầu vào có độ dài tùy chọn tối đa 255 ký tự.

Giải thích mã nguồn: Sau khi đọc chuỗi văn bản (tối đa 255 ký tự) và số bước dịch $n \in [0, 9]$, chương trình duyệt con trỏ `%edx` qua từng byte của bộ đệm để tiến hành mã hóa. Tại mỗi bước, nếu ký tự thuộc bảng chữ cái tiếng Anh, chương trình xác định mốc cơ sở (`'A'` cho chữ hoa hoặc `'a'` cho chữ thường) rồi đưa mã về khoảng giá trị $0..25$, cộng thêm n, và tự động xoay vòng bằng cách trừ đi 26 nếu bị tràn trước khi chuyển ngược lại mã ASCII gốc. Toàn bộ chữ số, khoảng trắng và ký tự đặc biệt được giữ nguyên vẹn mà không làm thay đổi định dạng văn bản gốc, sau đó kết quả được xuất ra màn hình qua `sys_write`.

Mã nguồn chương trình (`bai2.3.s`):

```
.section .rodata
enter_string:
    .ascii "Enter a string (<255 chars): "
enter_string_len = .-enter_string

shift_string:
    .ascii "Enter n (0-9): "
shift_string_len = .-shift_string

.section .bss
    .lcomm input_str, 256
    .lcomm input_len, 4
    .lcomm n, 2

.section .text
    .globl _start

printf:
    movl $0x1, %ebx         # stdout
    movl $0x4, %eax         # write
    int $0x80
    ret

read:
    xorl %ebx, %ebx         # stdin
    movl $0x3, %eax         # read
    int $0x80
    ret

cipher:
    movl $0, %ecx
    movl $input_str, %edx
    decl %edx

.cipher_loop:
    cmpl input_len, %ecx
    je .fin
    incl %edx
    incl %ecx
    cmpb $'A', (%edx)
    jb .cipher_loop     # Ky tu ngoai khoang (giu nguyen)
    cmpb $'Z', (%edx)
```

```
    movb $'A', %bl
    jbe .shift          # Chu hoa ('A' <= x <= 'Z')

    cmpb $'a', (%edx)
    jb .cipher_loop     # Ky tu dac biet nam giua 'Z' va 'a'
    cmpb $'z', (%edx)
    movb $'a', %bl
    jbe .shift          # Chu thuong ('a' <= x <= 'z')

    jmp .cipher_loop    # Ky tu lon hon 'z'

.fin:
    ret

.shift:
    movb (%edx), %al
    subb %bl, %al       # Dua ve khoang 0 .. 25
    addb n, %al         # Cong buoc dich n
    cmpb $26, %al       # Kiem tra tran 26
    jb .done
    subb $26, %al       # Xoay vong bang cach tru 26

.done:
    addb %bl, %al       # Dua ve lai ma ASCII goc
    movb %al, (%edx)    # Ghi de ky tu da ma hoa vao buffer
    jmp .cipher_loop

_start:
    movl $enter_string, %ecx
    movl $enter_string_len, %edx
    call printf

    movl $input_str, %ecx
    movl $256, %edx
    call read
    movl %eax, input_len

    movl $shift_string, %ecx
    movl $shift_string_len, %edx
    call printf

    movl $n, %ecx
    movl $0x2, %edx
    call read
    subb $0x30, n
    call cipher

    movl $input_str, %ecx
    movl input_len, %edx
    call printf

    movl $1, %eax
    int $0x80
```

1.4 Bài 2.4: Đếm số lượng bội của 10 trong khoảng [x, y]

Yêu cầu đề bài: Nhập từ bàn phím hai số nguyên có 2 chữ số x và y ($10 \leq x \leq y \leq 99$). Đếm số lượng các số chia hết cho 10 nằm trong đoạn đóng $[x, y]$ (tính cả x và y).

Giải thích mã nguồn: Thay vì sử dụng vòng lặp duyệt tốn kém, chương trình áp dụng công thức toán học $O(1)$ để tính số lượng bội của 10 trong đoạn đóng $[x, y]$ thông qua hiệu số $\lfloor y/10 \rfloor - \lfloor (x-1)/10 \rfloor$. Hai chuỗi 2 chữ số x và y nhập từ bàn phím được chuyển đổi sang số nguyên; x giảm đi 1 rồi chia 10 lấy thương lưu vào `%edi`, còn y chia 10 lấy thương lưu vào `%esi`. Hiệu số giữa hai thương được cộng thêm `'0'` thành ký tự ASCII, đẩy trực tiếp lên đỉnh ngăn xếp (`pushl %esi`) và tận dụng địa chỉ con trỏ đỉnh ngăn xếp `%esp` làm bộ nhớ đệm để xuất kết quả ra màn hình mà không cần cấp phát thêm biến.

Mã nguồn chương trình (`bai2.4.s`):

```
.section .data
hihi:
    .string "Nhap so: "
string_len = . - hihi
newline:
    .ascii "\n"

.section .bss
    .lcomm x, 3
    .lcomm y, 3

.section .text
.globl _start

_start:
    # 1. Nhap va tinh [(x - 1) / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80

    movl $0, %ebx
    movl $3, %eax
    movl $x, %ecx
    movl $3, %edx
    int $0x80

    movzbl x, %edi
    subl $'0', %edi
    imull $10, %edi

    movzbl x+1, %eax
    subl $'0', %eax
    addl %eax, %edi

    decl %edi            # Tinh x - 1
    movl $10, %ecx
    movl %edi, %eax
    xorl %edx, %edx
    divl %ecx            # Chia 10: thuong nam o %eax
    movl %eax, %edi      # Luu [(x - 1) / 10] vao %edi

    # 2. Nhap va tinh [y / 10]
    movl $4, %eax
    movl $1, %ebx
    movl $hihi, %ecx
    movl $string_len, %edx
    int $0x80

    movl $0, %ebx
    movl $3, %eax
    movl $y, %ecx
    movl $3, %edx
    int $0x80

    movzwl y, %eax
    subb $'0', %al
    subb $'0', %ah

    movzbl %al, %ecx
    imull $10, %ecx
    movzbl %ah, %edx
    addl %edx, %ecx      # y gio nam o %ecx

    movl %ecx, %eax
    movl $10, %ecx
    xorl %edx, %edx
    divl %ecx            # Chia 10: thuong o %eax
    movl %eax, %esi      # Luu [y / 10] vao %esi

    # 3. Tinh hieu so va xuat ket qua
    subl %edi, %esi      # %esi = [y/10] - [(x-1)/10]
```

```
    addl $'0', %esi      # Chuyen thanh ky tu ASCII
    pushl %esi           # Day vao dinh ngan xep

    movl $4, %eax
    movl $1, %ebx
    movl %esp, %ecx      # Dung dia chi %esp lam buffer xuat
    movl $1, %edx
    int $0x80

    movl $4, %eax        # Xuat dau xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80

    movl $1, %eax        # Thoat
    xorl %ebx, %ebx
    int $0x80
```

1.5 Bài 2.5: Kiểm tra tuổi đoàn dựa vào năm sinh (Cơ bản + Nâng cao)

Yêu cầu đề bài:

- **Cơ bản:** Nhập năm sinh có 4 chữ số từ bàn phím. Nếu tuổi từ 16 đến 30 tuổi thì xuất thông báo `"Thuoc Doan TN"`, ngược lại xuất `"Khong thuoc Doan TN"`.
- **Nâng cao:** Xuất kèm theo số tuổi cụ thể tính được (ví dụ: `"Tuoi: 21"`).

Giải thích mã nguồn: Chương trình nhận chuỗi năm sinh 4 chữ số, tính giá trị nguyên theo trọng số hàng nghìn, trăm, chục, đơn vị rồi lấy mốc năm 2026 trừ đi để xác định tuổi. Tiếp theo, lệnh `cmpl` rẽ nhánh kiểm tra: nếu tuổi nằm ngoài khoảng $[16, 30]$ thì in thông báo `"Khong thuoc Doan TN"`, ngược lại in `"Thuoc Doan TN"`. Đối với yêu cầu nâng cao, thủ tục `print` chia liên tiếp số tuổi cho 10, đẩy từng phần dư (mã ASCII) vào ngăn xếp theo cơ chế LIFO rồi lần lượt rút ra xuất ra màn hình, giúp hiển thị chính xác số tuổi nhiều chữ số theo đúng thứ tự mà không cần đảo chuỗi thủ công.

Mã nguồn chương trình (`bai2.5.s`):

```
.section .data
hello:
    .ascii "Nhap nam sinh: "
string_len = . - hello

Thuoc:
    .ascii "Thuoc Doan TN\n"
Thuoc_len = . - Thuoc

Khongthuoc:
    .ascii "Khong thuoc Doan TN\n"
Khongthuoc_len = . - Khongthuoc

intuoi:
    .ascii "Tuoi: "
intuoi_len = . - intuoi

newline:
    .ascii "\n"

.section .bss
    .lcomm tuoi, 5
    .lcomm out, 1

.section .text
.globl _start

_start:
```

```
    # 1. Nhap nam sinh tu ban phim
    movl $4, %eax
    movl $1, %ebx
    movl $hello, %ecx
    movl $string_len, %edx
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $tuoi, %ecx
    movl $5, %edx
    int $0x80

    # 2. Chuyen 4 ky tu thanh so nguyen trong %edi
    xorl %edi, %edi

    movzbl tuoi, %eax
    subl $'0', %eax
    imull $1000, %eax
    addl %eax, %edi

    movzbl tuoi+1, %eax
    subl $'0', %eax
    imull $100, %eax
    addl %eax, %edi

    movzbl tuoi+2, %eax
    subl $'0', %eax
    imull $10, %eax
    addl %eax, %edi

    movzbl tuoi+3, %eax
    subl $'0', %eax
    addl %eax, %edi

    # 3. Tinh tuoi: 2026 - nam_sinh
    movl $2026, %eax
    subl %edi, %eax
    movl %eax, %edi

    # 4. Kiem tra khoang [16, 30]
    cmpl $16, %edi
    jl ngoai_khoang
    cmpl $30, %edi
    jg ngoai_khoang

    movl $4, %eax
    movl $1, %ebx
    movl $Thuoc, %ecx
    movl $Thuoc_len, %edx
    int $0x80
    jmp nang_cao

ngoai_khoang:
    movl $4, %eax
    movl $1, %ebx
    movl $Khongthuoc, %ecx
    movl $Khongthuoc_len, %edx
    int $0x80

nang_cao:
    movl $4, %eax
    movl $1, %ebx
    movl $intuoi, %ecx
    movl $intuoi_len, %edx
    int $0x80
    call print

done:
    movl $1, %eax
    xorl %ebx, %ebx
    int  $0x80

```

```
# Thu tuc print: in so nguyen trong %edi ra man hinh
print:
    movl %edi, %eax
    xorl %esi, %esi

chia_loop:
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx            # Chia 10 lay so du vao %edx
    addb $'0', %dl       # Chuyen sang ma ASCII
    pushl %edx           # Day vao ngan xep
    incl %esi            # Tang bien dem chu so
    cmpl $0, %eax
    jne chia_loop

in_loop:
    popl %eax            # Rut chu so theo thu tu dung
    movb %al, out
    movl $4, %eax
    movl $1, %ebx
    movl $out, %ecx
    movl $1, %edx
    int $0x80
    decl %esi
    jnz in_loop

    movl $4, %eax        # In ky tu xuong dong
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret
```

1.6 Bài 2.6: Kiểm tra năm nhuận (Cơ bản + Nâng cao)

Yêu cầu đề bài:

- **Cơ bản:** Nhập một năm a có 4 chữ số ($1000 \leq a \leq 9999$). Xuất ra `"Nam nhuan"` nếu là năm nhuận, ngược lại xuất `"Nam khong nhuan"`.
- **Nâng cao:** Tìm và in ra năm nhuận kế tiếp sau năm vừa nhập.

Giải thích mã nguồn: Chương trình đọc năm 4 chữ số và kiểm tra tính nhuận theo quy tắc lịch Gregory: nếu hai chữ số cuối bằng `00` (năm tròn thế kỷ), chương trình chia cho 400; nếu khác `00`, chương trình chỉ cần chia 4 hai chữ số cuối để đưa ra kết luận `"Nam nhuan"` hoặc `"Nam khong nhuan"`. Tiếp đó, hàm `namnhuantieptheo` liên tục tăng giá trị năm lên 1 và lặp lại kiểm tra chia hết cho 4, 100 và 400 cho đến khi tìm được năm nhuận kế tiếp. Giá trị năm tìm được được chuyển đổi ngược thành chuỗi bằng cách chia 10 lấy dư và ghi lùi từ cuối bộ đệm `buffer_num` về trước rồi xuất ra màn hình.

Mã nguồn chương trình (`bai2_6.s`):

```
.section .data
nhapvao:
    .string  "Enter a year (4-digit): "
kqnamnhuan:
    .string "Nam nhuan\n"
kqnamkhongnhuan:
    .string "Nam khong nhuan\n"
namnhuantieptheouwu:
    .string "Nam nhuan ke tiep: "
newline:
    .byte 10
ans:
    .int 0

```

```
.section .bss
    .lcomm number1, 5
    buffer_num: .space 16

.section .text
    .globl _start

jmp _start

namnhuantieptheo:
    addl $1, ans         # Tang nam len 1
    movl ans, %eax
    xorl %edx, %edx
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Khong chia het cho 4 -> lap tiep

    movl ans, %eax
    xorl %edx, %edx
    movl $100, %ecx
    divl %ecx
    cmpl $0, %edx
    jne okroido          # Chia het cho 4 va khong chia het cho 100 -> Nam nhuan!

    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namnhuantieptheo # Chia het cho 100 nhung khong chia het cho 400 -> Lap tiep

okroido:
    movl $4, %eax
    movl $1, %ebx
    movl $namnhuantieptheouwu, %ecx
    movl $19, %edx
    int $0x80

    movl ans, %eax
    movl $buffer_num + 10, %esi
    movl $0, %ecx

chuyen_doi:
    decl %esi
    xorl %edx, %edx
    movl $10, %ebx
    divl %ebx
    addb $48, %dl
    movb %dl, (%esi)
    incl %ecx
    cmpl $0, %eax
    jne chuyen_doi

    movl $4, %eax
    movl $1, %ebx
    movl %esi, %ecx
    movl $buffer_num + 10, %edx
    subl %esi, %edx
    int $0x80

    call xuonghang
    ret

xuonghang:
    movl $4, %eax
    movl $1, %ebx
    movl $newline, %ecx
    movl $1, %edx
    int $0x80
    ret

_start:
```

```
    movl $4, %eax
    movl $1, %ebx
    movl $nhapvao, %ecx
    movl $23, %edx
    int $0x80

    movl $3, %eax
    movl $0, %ebx
    movl $number1, %ecx
    movl $5, %edx
    int $0x80

    # Chuyen 4 ky tu thanh so nguyen ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans

    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans

    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

    # Kiem tra 2 chu so cuoi co cung bang 0 hay khong
    movl $0, %ecx
    addl %eax, %ecx
    addl %ebx, %ecx
    cmpl $0, %ecx
    je tieptuckiemtra
    jmp saudo

tieptuckiemtra:          # Nam the ky: kiem tra chia het cho 400
    movl ans, %eax
    xorl %edx, %edx
    movl $400, %ecx
    divl %ecx
    cmpl $0, %edx
    jne namkhongnhuan

lanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

namkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80
    jmp end

saudo:                   # Nam binh thuong: kiem tra 2 chu so cuoi chia het cho 4
    movl $0, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans

    movzbl number1+3, %ebx
    subl $48, %ebx
```

```
    addl %ebx, ans

    movl ans, %eax
    xorl %edx, %edx
    movl $4, %ecx
    divl %ecx
    cmpl $0, %edx
    jne tnamkhongnhuan

tlanamnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamnhuan, %ecx
    movl $10, %edx
    int $0x80
    jmp end

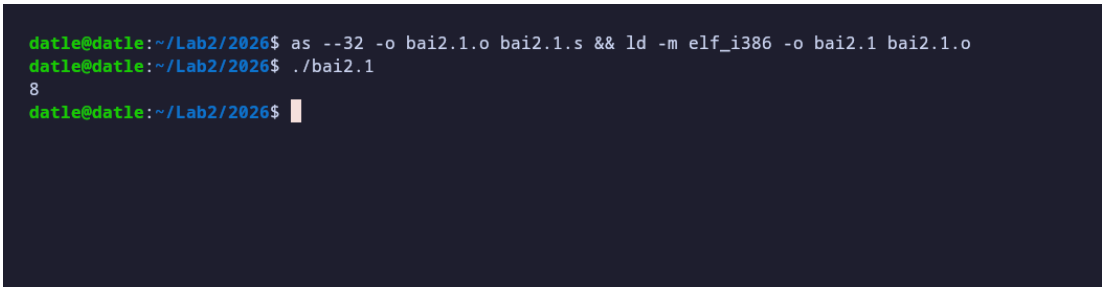
tnamkhongnhuan:
    movl $4, %eax
    movl $1, %ebx
    movl $kqnamkhongnhuan, %ecx
    movl $16, %edx
    int $0x80

end:
    # Tinh lai gia tri nam goc va tim nam nhuan tiep theo
    movl $0, ans
    movzbl number1, %eax
    subl $48, %eax
    imull $1000, %eax
    addl %eax, ans
    movzbl number1+1, %eax
    subl $48, %eax
    imull $100, %eax
    addl %eax, ans
    movzbl number1+2, %eax
    subl $48, %eax
    imull $10, %eax
    addl %eax, ans
    movzbl number1+3, %ebx
    subl $48, %ebx
    addl %ebx, ans

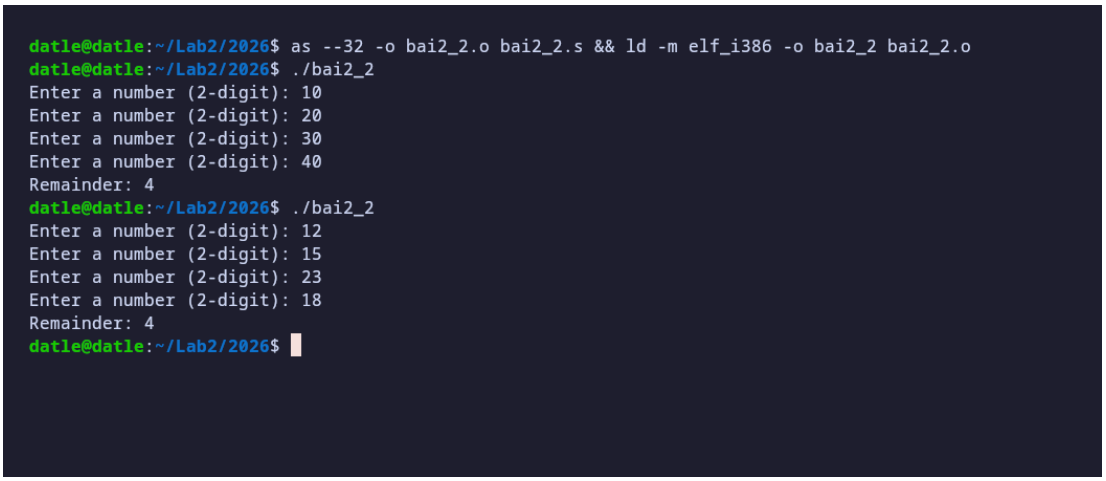
    call namnhuantieptheo

    mov $1, %eax
    int $0x80
```

2 Minh chứng kết quả thực thi



Hình 1: Kết quả biên dịch, liên kết và thực thi Bài 2.1 (In độ dài chuỗi cho trước).



Hình 2: Kết quả biên dịch, liên kết và thực thi Bài 2.2 (Số dư của tổng 4 số khi chia 8).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2.3.o bai2.3.s && ld -m elf_i386 -o bai2.3 bai2.3.o
datle@datle:~/Lab2/2026$ ./bai2.3
Enter a string (<255 chars): Hello World, XYZ xyz!
Enter n (0-9): 3
Khoor Zruog, ABC abc!
datle@datle:~/Lab2/2026$ ./bai2.3
Enter a string (<255 chars): UIT Assembly 2026
Enter n (0-9): 5
ZNY Fxxjrgqd 2026
datle@datle:~/Lab2/2026$
```

Hình 3: Kết quả biên dịch, liên kết và thực thi Bài 2.3 (Mã hóa Caesar nâng cao 1 & 2).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2.4.o bai2.4.s && ld -m elf_i386 -o bai2.4 bai2.4.o
datle@datle:~/Lab2/2026$ ./bai2.4
Nhap so: 10
Nhap so: 30
3
datle@datle:~/Lab2/2026$ ./bai2.4
Nhap so: 15
Nhap so: 45
3
datle@datle:~/Lab2/2026$ ./bai2.4
Nhap so: 21
Nhap so: 29
0
datle@datle:~/Lab2/2026$
```

Hình 4: Kết quả biên dịch, liên kết và thực thi Bài 2.4 (Đếm số lượng bội của 10 trong đoạn $[x, y]$).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2.5.o bai2.5.s && ld -m elf_i386 -o bai2.5 bai2.5.o
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2005
Thuoc Doan TN
Tuoi: 21
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 1990
Khong thuoc Doan TN
Tuoi: 36
datle@datle:~/Lab2/2026$ ./bai2.5
Nhap nam sinh: 2015
Khong thuoc Doan TN
Tuoi: 11
datle@datle:~/Lab2/2026$
```

Hình 5: Kết quả biên dịch, liên kết và thực thi Bài 2.5 (Kiểm tra tuổi đoàn và in kèm số tuổi).

```
datle@datle:~/Lab2/2026$ as --32 -o bai2_6.o bai2_6.s && ld -m elf_i386 -o bai2_6 bai2_6.o
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2024
Nam nhuan
Nam nhuan ke tiep: 2028
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2023
Nam khong nhuan
Nam nhuan ke tiep: 2024
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 1900
Nam khong nhuan
Nam nhuan ke tiep: 1904
datle@datle:~/Lab2/2026$ ./bai2_6
Enter a year (4-digit): 2000
Nam nhuan
Nam nhuan ke tiep: 2004
datle@datle:~/Lab2/2026$
```

Hình 6: Kết quả biên dịch, liên kết và thực thi Bài 2.6 (Kiểm tra năm nhuận và tìm năm nhuận kế tiếp).